IE 7615 DEEP LEARNING FOR AI · DISCRIMINATIVE PROJECT

Milestone 2: Multi-Object Detection with YOLOv8

Detecting, identifying and locating the class's 73 objects in multi-object images

Study Group 6: Sankalp Hegde · Reshma Jose · Jairaghavendra Sridhar

0.971	0.963	0.964 / 0.941	2,600
test mAP@0.5 (YOLOv8s)	test mAP@0.5:0.95	precision / recall	generated multi-object images

1 Problem Overview

Milestone 1 identified a single object per image. Milestone 2 extends this to **object detection**: given one image containing several of the class's 73 objects, the model must report the **Object ID and the location (bounding box)** of every object. We generate a multi-object dataset by compositing randomly selected single-object images from the final merged class dataset, and fine-tune a COCO-pretrained YOLOv8 detector on it (transfer learning). Object IDs OBJ001–OBJ073 match the final Object ID sheet; YOLO class k = OBJ$k{+}1$.

2 Multi-Object Dataset Generation

Each generated image is a 640×640 canvas with **2–6 distinct objects** pasted onto a background. Because our script places every object, its bounding box is known exactly and the YOLO label file is written automatically; no manual annotation was needed.

- **Object selection:** objects are drawn at random from the cleaned single-object dataset of Milestone 1 (6,954 images, 73 classes; background-only `*_bg_*` photos are never used as objects).
- **No train/test leakage:** training composites use only the Milestone 1 *training* photos, validation composites only *validation* photos and test composites only *test* photos, so the detector is always evaluated on photos it never saw.
- **Backgrounds:** 60% use the class's background-only photos (rooms, floors, desks, outdoors); the rest use plain colours or blurred noise textures.
- **Layouts:** 70% random placement (overlap limited to IoU ≤ 0.1, so partial occlusion occurs), 30% jittered 2×2 / 3×3 grids.
- **Augmentation per object:** random size (120–270 px), horizontal flip, brightness / contrast / saturation jitter, occasional blur and ±15° rotation; plus whole-image brightness jitter. YOLOv8's own training augmentation (mosaic, HSV, flips, scale/translate) is applied on top.

Split	Images	Objects	Objects / image
train	2,000	7,695	3.8
val	300	1,166	3.9
test	300	1,226	4.1

Every object appears in all three splits; in the training set each object has 79–131 instances. The assignment asks each student for at least 100 multi-object images; together the group generated 2,600.

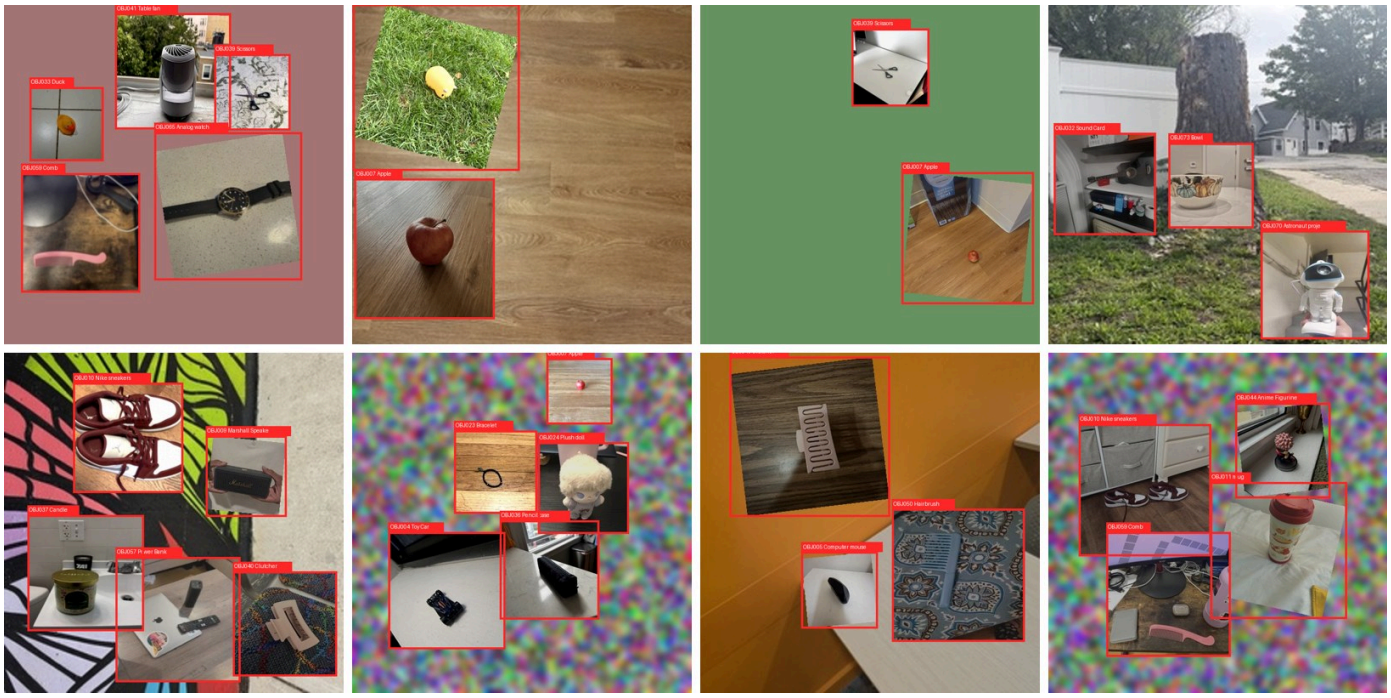


Figure 1. Generated training images with their automatically written ground-truth boxes.

3 Model and Training

We fine-tuned two sizes of **YOLOv8** (Ultralytics), both initialised from COCO-pretrained weights, so the backbone already knows generic object features and only has to learn our 73 objects. The detection head was re-initialised for 73 classes.

Setting	Value
Models	YOLOv8n (nano) and YOLOv8s (small), COCO-pretrained
Input size / batch	640 × 640 / 16
Epochs	up to 60 (n) and 40 (s), early stopping patience 15, best weights by validation fitness
Optimizer	Ultralytics default (auto-selected optimizer and learning-rate schedule, warm-up)
Augmentation	mosaic (off for the last 10 epochs), HSV jitter, horizontal flip, scale/translate
Hardware	Apple M4 GPU (PyTorch MPS)

4 Results

Model	Precision	Recall	mAP@0.5	mAP@0.5:0.95	Params	Epochs	Train (min)	ms / image
YOLOv8s ★	0.964	0.941	0.971	0.963	11.2M	40	164	2.4
YOLOv8n	0.929	0.882	0.937	0.922	3.1M	60	104	1.1

★ = selected model. Metrics are on the held-out test set (300 images, 1,226 objects) built from Milestone 1 test photos only.

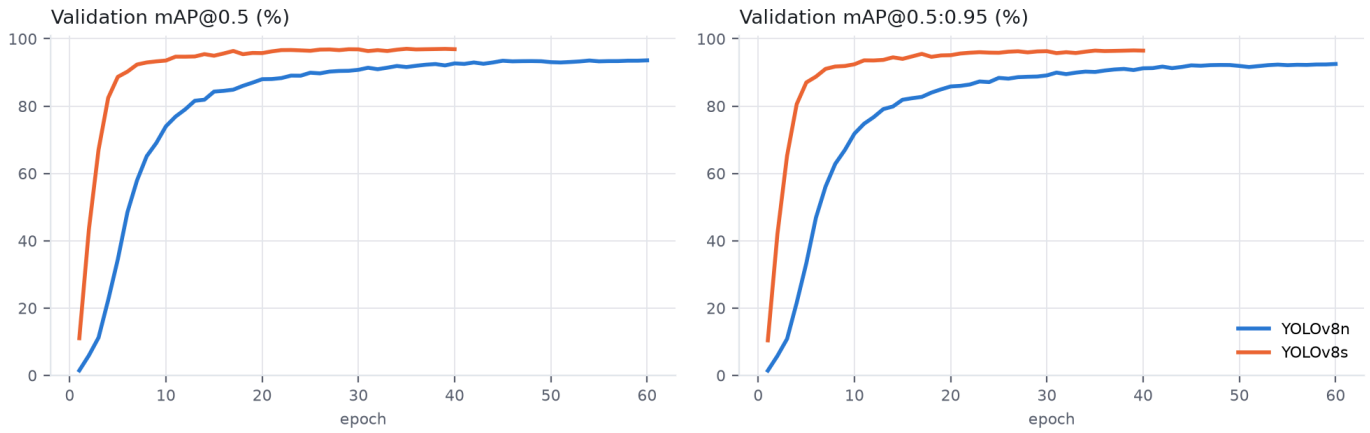


Figure 2. Validation mAP per epoch.

Model size. YOLOv8s has 3.6× the parameters of YOLOv8n. On the test set it scores mAP@0.5:0.95 0.963 vs. 0.922 (+4.1 points) and mAP@0.5 0.971 vs. 0.937. Inference takes 2.4 ms vs. 1.1 ms per image on our hardware.

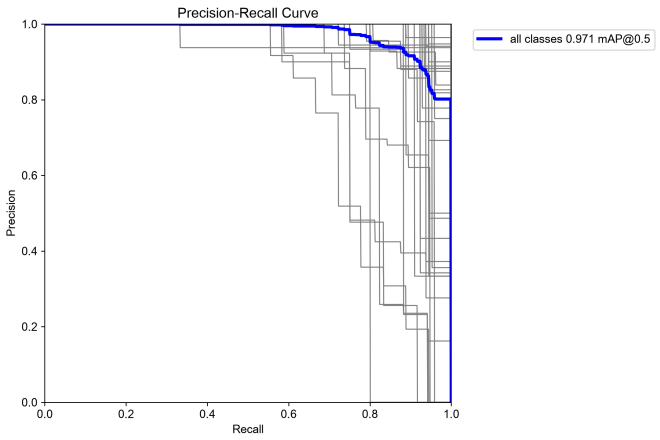


Figure 3. Precision–recall curve, YOLOv8s (test).

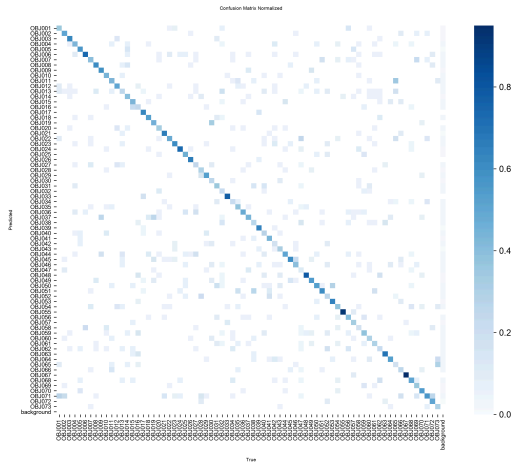


Figure 4. Normalised confusion matrix, YOLOv8s (test).

5 Detecting IDs and Locations

Given a new multi-object image, `detect_objects.py` returns every detected object's Object ID, name, confidence and bounding box. Example output for test image `test_00003.jpg` (all ground-truth objects found):

Object ID	Name	Confidence	Top-left (x, y)	Bottom-right (x, y)
OBJ044	Anime Figurine	0.98	(19, 47)	(168, 196)
OBJ063	Measuring Bottle	0.99	(252, 219)	(419, 385)
OBJ009	Marshall Speaker	0.98	(41, 236)	(183, 378)
OBJ025	Watch	0.99	(437, 250)	(612, 425)
OBJ039	Scissors	0.96	(220, 434)	(420, 636)
OBJ041	Table fan	0.98	(473, 434)	(639, 600)

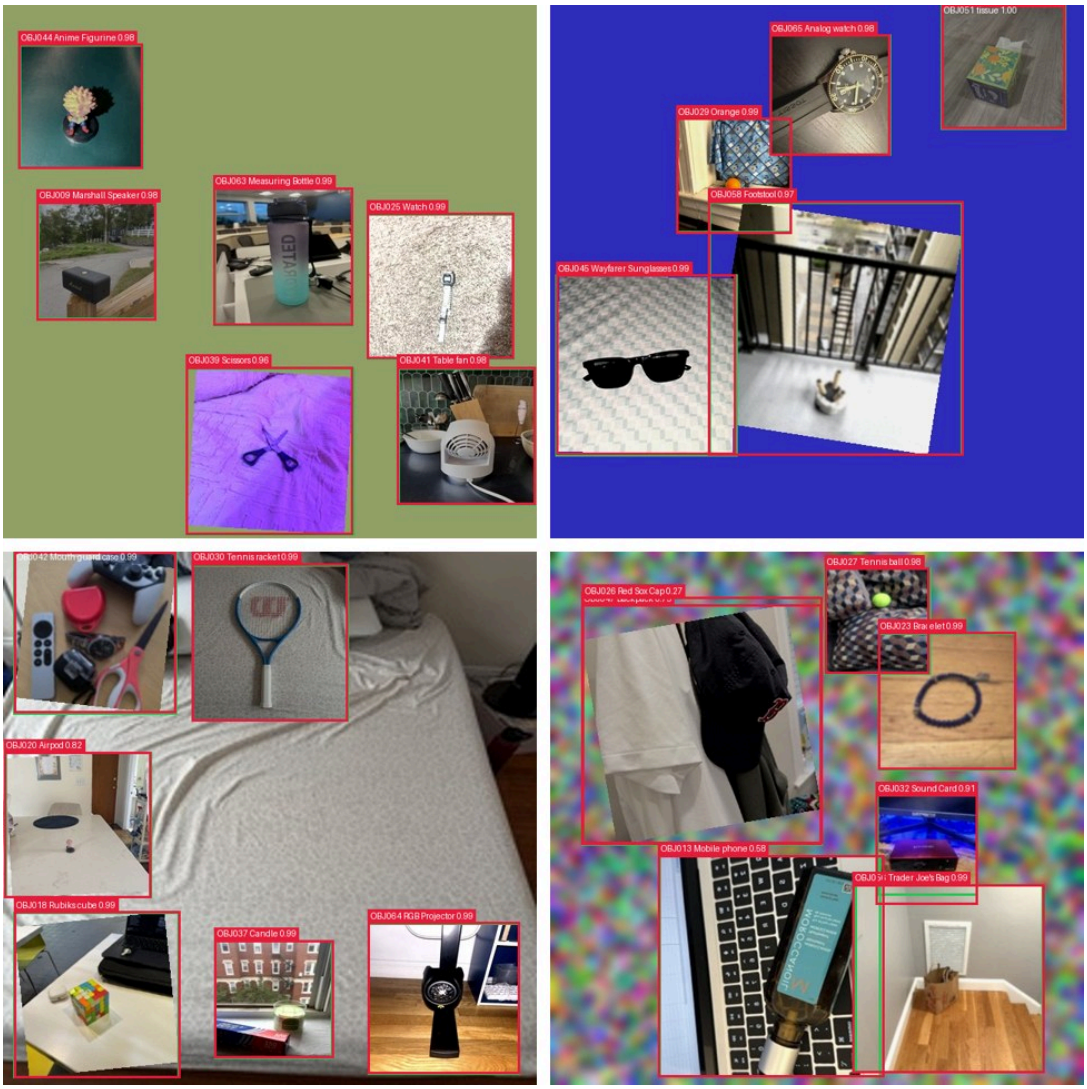


Figure 5. Test-set detections by YOLOv8s (red = prediction with ID, name and confidence; green = ground truth).

6 Analysis

Per-object accuracy. 57 of 73 objects reach a test mAP@0.5:0.95 of at least 0.95. The weakest objects are OBJ034 (Table Tennis Racket) 0.75, OBJ052 (Spoon) 0.79, OBJ060 (Sleep Mask) 0.80, OBJ061 (Card wallet) 0.82, OBJ013 (Mobile phone) 0.85, OBJ058 (Footstool) 0.88.

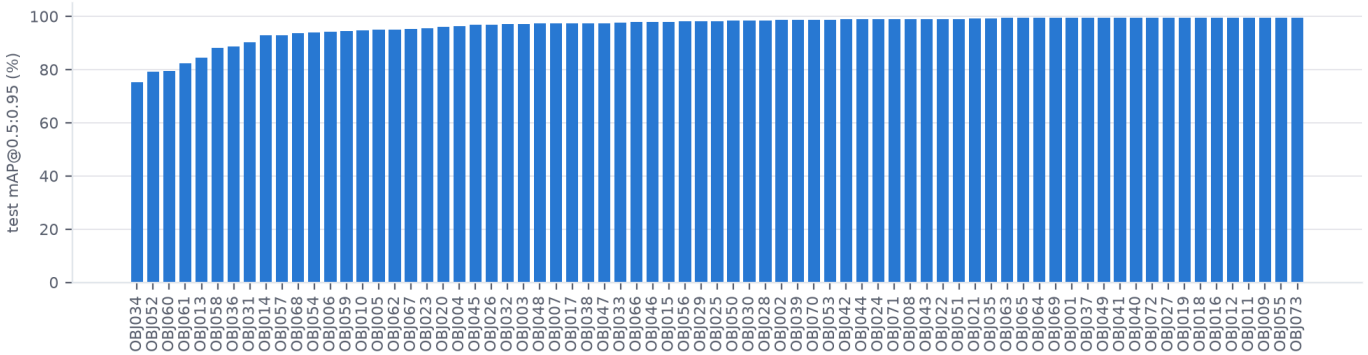


Figure 6. Test mAP@0.5:0.95 per object, YOLOv8s.

Localisation. mAP@0.5 is close to mAP@0.5:0.95 because each ground-truth box is the full pasted photo, a crisp rectangle that is easy to localise precisely. Most remaining errors are *identification* errors between similar-looking objects, or objects that are small or partly covered by another photo.

Transfer learning. Starting from COCO weights, the detector passed mAP@0.5 = 0.93 on validation within 8 epochs.

7 Final Model Selection

We select **YOLOv8s**: the highest test mAP@0.5:0.95 (0.963) and mAP@0.5 (0.971), with precision 0.964 and recall 0.941, at 2.4 ms per image.

8 Limitations and Next Steps

- **Box definition:** each label covers the whole pasted photo, not a tight box around the object inside it, so the model learns to find object *photos*. Tight boxes would need manual annotation or a segmentation step.
- **Synthetic scenes:** composites have sharp tile edges and no real lighting interaction; performance on real photos with several physical objects will be lower. Testing on real multi-object photos is the natural next step.
- **Similar objects** (e.g. two kinds of sunglasses, two watches) remain the main source of identification errors.

9 Code Repository

All code (dataset generator, training, evaluation, detection script) and trained weights are on GitHub: https://github.com/sankalphegde/ie7615-discriminative-project (folder `milestone2/`).